

به نام خدا

دوره ی آموزش برنامه نویسی به زبان C#.NET



علی عظیم زاده



ali-azimzadeh



0919-2757749



@AcademyOfComputer



ali-azimzadeh-



aliazimzadeh@gmail.com



@Academy_Of_Computer

✓ مشاور نرم افزار

✓ مهندس نرم افزار

✓ مدرس برنامه نویسی

✓ طراح و برنامه نویس

✓ عضو سازمان نظام صنفی رایانه ای

دات نت چیست ؟

دات نت یک پلت فرم توسعه دهنده منبع باز است که توسط مایکروسافت برای ساخت انواع مختلف برنامه ها ایجاد شده است.

دات نت یک پلتفرم توسعه دهنده رایگان، چند پلتفرمی و منبع باز برای ساخت انواع مختلف برنامه های کاربردی است.

با دات نت، می توانید از چندین زبان، ویرایشگر و کتابخانه برای ساخت وب، موبایل، دسکتاپ، بازی ها و اینترنت اشیا استفاده کنید.

زبان های .NET :

می توانید برنامه های دات نت را به زبان های C# ، F# یا Visual Basic و چندین زبان دیگر بنویسید. C# یک زبان برنامه نویسی ساده، مدرن، شی گرا و ایمن است.

F# یک زبان برنامه نویسی است که نوشتن کد مختصر، قوی و عملکردی را آسان می کند.

Visual Basic یک زبان قابل دسترسی با یک نحو ساده برای ساخت برنامه های شی گرا است.

مستقل از پلت فرم

فرقی نمی کند که در C# ، F# یا Visual Basic کار می کنید، کد شما به صورت بومی روی هر سیستم عاملی سازگار است و اجرا می شود

- **دات نت** یک پیاده سازی چند پلتفرمی برای وب سایت ها، سرورها و برنامه های کنسول در ویندوز، لینوکس و macOS است.

- [.NET Framework](#) از وب سایت ها، خدمات، برنامه های دسکتاپ و موارد دیگر در ویندوز پشتیبانی می کند.

- [Xamarin/Mono](#) یک پیاده سازی دات نت برای اجرای برنامه ها در تمام سیستم عامل های اصلی تلفن همراه است.

مدل های کاربردی

شما می توانید انواع مختلفی از برنامه ها را با دات نت بسازید. برخی از آنها چند پلتفرمی هستند و برخی دیگر یک سیستم عامل خاص یا اجرای دات نت را هدف قرار می دهند.



وب

برنامه ها و سرویس های وب
برای Windows, Linux,
macOS و Docker بسازید.



سیار

از یک پایگاه کد واحد برای
ساخت برنامه های تلفن
iOS، همراه بومی برای
Android و Windows
استفاده کنید.



دسکتاپ

برنامه های دسکتاپ زیبا و
macOS جذاب برای ویندوز و
ایجاد کنید.



میکروسرویس ها

میکروسرویس های مستقل
قابل استقرار ایجاد کنید که
اجرا Docker روی کانتینرهای
می شوند.



ابر

خدمات ابری موجود را
مصرف کنید یا خدمات خود را
ایجاد و اجرا کنید.



فراگیری ماشین

الگوریتم های بینایی، پردازش
گفتار، مدل های پیش بینی و
موارد دیگر را به برنامه های
خود اضافه کنید.



توسعه بازی

بازی های دو بعدی و سه
بعدی را برای محبوب ترین
رایانه های رومیزی، تلفن ها و
کنسول ها توسعه دهید.



اینترنت اشیا

با پشتیبانی بومی برای
Raspberry Pi و سایر رایانه
IoT های تک برد، برنامه های
بسازید.

چرا دات نت را انتخاب کنید؟

دات نت به شما کمک می کند تا برنامه های کاربردی با کیفیت بالا را سریعتر توسعه دهید. ساختارهای زبان مدرن مانند Generic، جستجوی یکپارچه زبان (LINQ) و برنامه نویسی ناهمزمان (Async)، توسعه دهندگان را پشتیبانی می کند.

به دلیل وجود کتابخانه های کلاس گسترده، API های رایج، پشتیبانی چند زبانه، و ابزار قدرتمند ارائه شده توسط [خانواده ویژوال استودیو](#)، دات نت پربازده ترین پلت فرم برای توسعه دهندگان است.

هر برنامه، در هر پلتفرمی

با دات نت می توانید هر نوع برنامه ای را که بر روی هر پلتفرمی اجرا می شود، توسعه دهید. توسعه دهندگان می توانند از مهارت ها و کدهای همه آنها در یک محیط دوباره استفاده کنند. این بدان معناست که توسعه دهندگان می توانند برنامه ها را سریعتر و با هزینه کمتر تولید کنند.

از برنامه های موبایلی که در iOS، Android و Windows اجرا می شوند، گرفته تا برنامه های کاربردی سرور Enterprise که روی Windows Server و Linux اجرا می شوند و یا میکروسرویس های با مقیاس بالا که در فضای ابری اجرا می شوند، دات نت راه حلی را برای شما فراهم می کند.

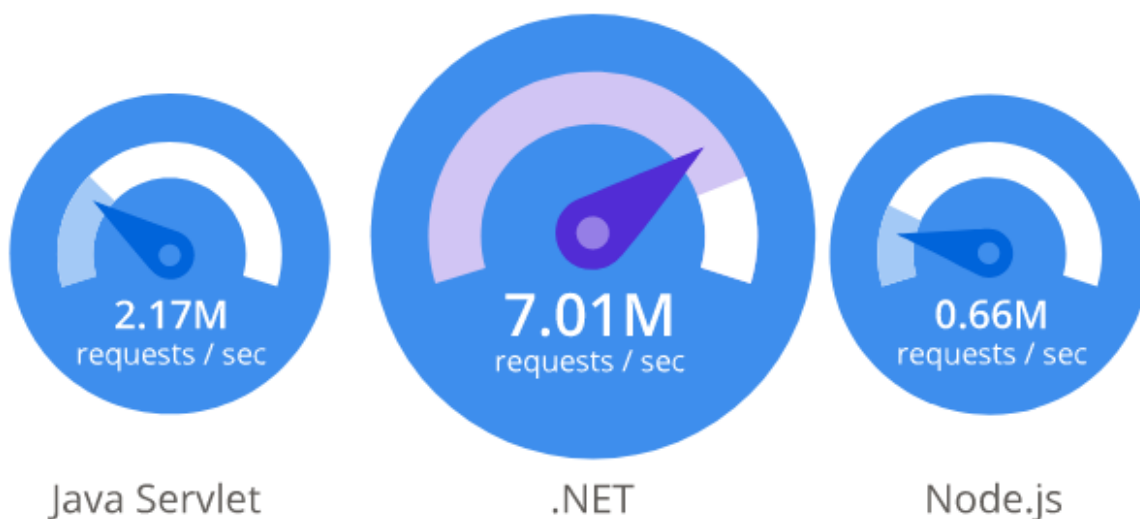
مورد علاقه توسعه دهندگان

دات نت یک پلتفرم توسعه مدرن، نوآورانه و متن باز است و توسعه دهندگان آن را دوست دارند. دات نت در هر دو نسخه [۲۰۱۹](#) و [۲۰۲۰](#) به عنوان محبوب ترین فریم ورک در نظرسنجی Stack Overflow Developer Survey رتبه اول را کسب کرد.

عملکرد در جایی که اهمیت دارد

دات نت سریع است. واقعا سریع! این بدان معناست که برنامه ها زمان پاسخگویی بهتری را ارائه می دهند و به توان محاسباتی کمتری نیاز دارند.

معیار محبوب [TechEmpower](#) چارچوب های برنامه های کاربردی وب را با کارهایی مانند سریال سازی JSON، دسترسی به پایگاه داده و ارائه الگوی سمت سرور مقایسه می کند .NET - سریعتر از هر فریم ورک محبوب دیگری عمل می کند.



قابل اعتماد و امن

پلت فرم دات نت رسما توسط مایکروسافت پشتیبانی می شود و هزاران شرکت و میلیون ها توسعه دهنده به آن اعتماد دارند. مایکروسافت امنیت را بسیار جدی می گیرد و در صورت کشف تهدید، به روزرسانی ها را به سرعت منتشر می کند.

سی شارپ - کلاس ها و اشیا چیست؟

کلاس ها و اشیا دو جنبه اصلی برنامه نویسی شی گرا هستند. برای مشاهده تفاوت بین کلاس و اشیا به تصویر زیر نگاه کنید:

کلاس	اشیا
میوه	سیب
	موز
	انبه

و مثالی دیگر :

کلاس	اشیا
ماشین	ولو
	آئودی
	تویوتا

بنابراین، یک کلاس یک الگو برای اشیا است، و یک شی نمونه ای از یک کلاس است. هنگامی که اشیا منفرد ایجاد می شوند، تمام متغیرها و متدها را از کلاس به ارث می برند.

کلاس ها و اشیاء سی شارپ

کلاس ها و اشیاء

تا الان حتماً متوجه شدید که سی شارپ یک زبان برنامه نویسی شی گرا است. همه چیز در سی شارپ با کلاس ها و اشیاء همراه با ویژگی ها و متدهای آن مرتبط است. به عنوان مثال: در زندگی واقعی، ماشین یک شی است. خودرو دارای ویژگی هایی مانند وزن و رنگ و **روش** هایی مانند رانندگی و ترمز و ... است.

یک کلاس مانند یک سازنده شی یا یک "طرح" برای ایجاد اشیا است.

حال چگونه یک کلاس ایجاد کنید :

در زبان سی شارپ برای ایجاد کلاس از کلمه کلیدی class استفاده کنید :

یک کلاس به نام "Car" با یک متغیر color ایجاد کنید :

```
class Car
{
    string color = "red";
}
```

هنگامی که یک متغیر مستقیماً در یک کلاس اعلام می شود، اغلب به عنوان یک **فیلد** یا ویژگی نامیده می شود.

بهتر است شروع نام کلاس ها با حرف اول بزرگ باشد. همچنین، معمولاً نام فایل سی شارپ و کلاس مطابقت دارند، زیرا کد ما را سازماندهی می کند. با این حال نیازی به آن نیست (مانند جاوا).

حال چگونه یک شی بسازید:

یک شی ، از یک کلاس ایجاد می شود. ما قبلاً کلاسی با نام **Car** ایجاد کرده ایم ، بنابراین اکنون می توانیم از آن برای ایجاد اشیا استفاده کنیم.

برای ایجاد یک شی از **Car**، نام کلاس و به دنبال آن نام شی را مشخص کنید و از کلمه کلیدی **new** استفاده کنید :

مثال: یک شی به نام "myObj" ایجاد کنید و از آن برای چاپ مقدار **color** استفاده کنید:

```
class Car
{
    string color = "red";

    static void Main(string[] args)
    {
        Car myObj = new Car();
        Console.WriteLine(myObj.color);
    }
}
```

سطوح دسترسی (Access Modifiers)

در **C#**، **سطوح دسترسی**، کلمات کلیدی هستند که برای تعریف سطح دسترسی برای همه نوع و اعضای کلاس به کار می روند. با تعیین سطح دسترسی برای همه اعضای کلاس، می توانیم بر اساس نیازهایمان کنترل کنیم که آیا می توان به کلاس های دیگر یا پروژه ی فعلی یا سایر مجموعه ها دسترسی داشت یا خیر

در زیر انواع مختلف سطوح دسترسی موجود در زبان برنامه نویسی **C#** آورده شده است.

- عمومی - public
- خصوصی - private
- حفاظت شده - protected
- درونی؛ داخلی - internal

با استفاده از این چهار تغییر دهنده دسترسی، می‌توانیم شش سطح دسترسی زیر را برای همه انواع و اعضای نوع بر اساس نیاز خود مشخص کنیم.

نحوه دسترسی	شرح
عمومی Public	برای تعیین اینکه دسترسی محدود نیست استفاده می‌شود.
خصوصی Private	برای تعیین اینکه دسترسی محدود است استفاده می‌شود.
حفاظت شده Protected	برای تعیین اینکه دسترسی به عضو یا نوع مشتق شده از کلاس محدود است استفاده می‌شود.
درونی؛ داخلی internal	برای تعیین اینکه دسترسی به مجموعه فعلی محدود است استفاده می‌شود.
داخلی محافظت شده Internal protected	مشخص می‌کند که دسترسی به مجموعه فعلی یا انواع مشتق شده از کلاس محدود است.

نحوه دسترسی	شرح
خصوصی محافظت شده Private protected	برای تعیین اینکه دسترسی محدود به <u>کلاس</u> یا انواعی است که از <u>کلاس</u> در مجموعه فعلی مشتق شده اند استفاده می شود.

به طور کلی، در سی شارپ تنها یک دسترسی مجاز به استفاده با هر عضو یا نوع است، به جز زمانی که از ترکیب های **محافظت شده داخلی** یا **خصوصی محافظت شده** استفاده می کنیم.

در سی شارپ، ما مجاز به استفاده از هیچ تغییر دهنده دسترسی در فضای نام (namespace) نیستیم زیرا فضاهای نام محدودیت دسترسی ندارند.

سطح دسترسی public :

در سی شارپ، سطح دسترسی **عمومی** برای تعیین اینکه دسترسی محدود نیست استفاده می شود، بنابراین نوع یا عضو تعریف شده را می توان با هر کد دیگری در اسمبلی فعلی یا اسمبلی دیگری که به آن ارجاع می دهد، دسترسی داشت.

در زیر مثالی از تعریف اعضا با تغییر دهنده **عمومی** در زبان برنامه نویسی C# آورده شده است.

```
class User
{
    public string Name;
    public string Location;
    public int Age;
    public void GetUserDetails()
    {
        Console.WriteLine("Name: {0}", Name);
        Console.WriteLine("Location: {0}", Location);
    }
}
```

```

        Console.WriteLine("Age: {0}", Age);
    }
}
class Program
{
    static void Main(string[] args)
    {
        User u = new User();
        u.Name = "Suresh Dasari";
        u.Location = "Hyderabad";
        u.Age = 32;
        u.GetUserDetails();
        Console.WriteLine("\nPress Enter Key to Exit..");
        Console.ReadLine();
    }
}

```

اگر به مثال بالا توجه کنید، ما یک کلاس User را با متغیرها و متدهای مورد نیاز با استفاده از دسترسی عمومی تعریف کرده ایم و سعی می کنیم به آن متغیرها و متد در کلاس دیگری با مرجع شیء کلاس User دسترسی پیدا کنیم . همانطور که بحث شد، مشخص کننده دسترسی عمومی ، تمام اعضا یا انواع تعریف شده را در دسترس همه انواع برنامه ها قرار می دهد.

سطح دسترسی private :

در سی شارپ، سط دسترسی خصوصی برای تعیین اینکه دسترسی به اعضا محدود است استفاده می شود، بنابراین نوع یا عضو تعریف شده تنها با کد موجود در همان کلاس یا ساختار قابل دسترسی است.

در زیر مثالی از تعریف اعضا با یک اصلاح کننده خصوصی در زبان برنامه نویسی C# آورده شده است.

```
class User
{
    private string Name;
    private string Location;
    private int Age;
    private void GetUserDetails()
    {
        Console.WriteLine("Name: {0}", Name);
        Console.WriteLine("Location: {0}", Location);
        Console.WriteLine("Age: {0}", Age);
    }
}

class Program
{
    static void Main(string[] args)
    {
        User u = new User();
        // Compiler Error
        // These are inaccessible due to private specifier
        u.Name = "Suresh Dasari";
        u.Location = "Hyderabad";
        u.Age = 32;
        u.GetUserDetails();
        Console.WriteLine("\nPress Enter Key to Exit..");
        Console.ReadLine();
    }
}
```

اگر به مثال بالا توجه کنید، ما یک **کلاس User** را با **متغیرها** و **متدهای** مورد نیاز با استفاده از دسترسی **خصوصی** تعریف کرده‌ایم و سعی می‌کنیم به آن **متغیرها** و **متد** در **کلاس** دیگری با مرجع شی از **کلاس User** دسترسی پیدا کنیم.

هنگامی که برنامه C# فوق را اجرا می‌کنید، با خطاهای زمان کامپایل مواجه خواهید شد.

سطح دسترسی **protected** :

در سی شارپ، اصلاح‌کننده **محافظت‌شده** برای تعیین اینکه دسترسی به نوع یا انواع مشتق‌شده از **کلاس** محدود است، استفاده می‌شود، بنابراین نوع یا عضو فقط با کد در همان **کلاس** یا **کلاس** مشتق‌شده قابل دسترسی است.

در زیر مثالی از تعریف اعضا با یک اصلاح‌کننده **محافظت‌شده** در زبان برنامه نویسی C# آورده شده است.

```
class User
{
    protected string Name;
    protected string Location;
    protected int Age;
    protected void GetUserDetails()
    {
        Console.WriteLine("Name: {0}", Name);
        Console.WriteLine("Location: {0}", Location);
        Console.WriteLine("Age: {0}", Age);
    }
}

class Program
{
    static void Main(string[] args)
```

```

{
    User u = new User();
    // Compiler Error
    // These are inaccessible due to protected specifier
    u.Name = "Suresh Dasari";
    u.Location = "Hyderabad";
    u.Age = 32;
    u.GetUserDetails();
    Console.WriteLine("\nPress Enter Key to Exit..");
    Console.ReadLine();
}
}

```

اگر به مثال بالا توجه کنید، ما یک **کلاس User** را با **متغیرها** و **متدهای** مورد نیاز با استفاده از اصلاح کننده دسترسی **محافظت شده** تعریف کرده ایم و سعی می کنیم به آن **متغیرها** و **متد** در **کلاس** دیگری با مرجع شی از **کلاس User** دسترسی پیدا کنیم.

هنگامی که برنامه C# فوق را اجرا می کنید، با خطاهای زمان کامپایل مواجه خواهید شد. در زیر مثالی از دسترسی به اعضای **محافظت شده کلاس** پایه در **کلاس** مشتق شده از طریق نوع **کلاس** مشتق شده آورده شده است.

```

class User
{
    protected string Name;
    protected string Location;
    protected int Age;
    protected void GetUserDetails()
    {
        Console.WriteLine("Name: {0}", Name);
        Console.WriteLine("Location: {0}", Location);
    }
}

```

```

        Console.WriteLine("Age: {0}", Age);
    }
}
class Program: User
{
    static void Main(string[] args)
    {
        User u = new User();
        Program p = new Program();
        // Compiler Error
        // protected members can only accessible with derived classes
        //u.Name = "Suresh Dasari";
        p.Name = "Suresh Dasari";
        p.Location = "Hyderabad";
        p.Age = 32;
        p.GetUserDetails();
        Console.WriteLine("\nPress Enter Key to Exit..");
        Console.ReadLine();
    }
}

```

سطح دسترسی internal :

در C#، سطح دسترسی داخلی برای تعیین اینکه دسترسی به پروژه ی فعلی محدود است استفاده می شود. نوع یا عضو با هر کدی در همان اسمبلی قابل دسترسی است اما از اسمبلی دیگر قابل دسترسی نیست.

در زیر مثالی از تعریف اعضا با یک اصلاح کننده داخلی در زبان برنامه نویسی C# آورده شده است.

```

class User
{
    internal string Name;
}

```

```

internal string Location;
internal int Age;
internal void GetUserDetails()
{
    Console.WriteLine("Name: {0}", Name);
    Console.WriteLine("Location: {0}", Location);
    Console.WriteLine("Age: {0}", Age);
}
}
class Program
{
    static void Main(string[] args)
    {
        User u = new User();
        u.Name = "Suresh Dasari";
        u.Name = "Suresh Dasari";
        u.Location = "Hyderabad";
        u.Age = 32;
        u.GetUserDetails();
        Console.WriteLine("\nPress Enter Key to Exit..");
        Console.ReadLine();
    }
}

```

اگر به مثال بالا توجه کنید، ما یک کلاس User را با متغیرها و متدهای مورد نیاز با استفاده از دسترس داخلی تعریف کرده ایم و سعی می کنیم به آن متغیرها و متد در کلاس دیگری با مرجع شی از کلاس User دسترسی پیدا کنیم.

سطح دسترسی **protected internal** :

در سی شارپ، سطح دسترسی **داخلی محافظت شده** برای تعیین اینکه دسترسی به پروژه ی فعلی یا انواع مشتق شده از کلاس محدود است استفاده می شود . عضو را می توان با هر کدی در همان اسمبلی یا هر کلاس مشتق شده در اسمبلی دیگر دسترسی داشت.

در زیر مثالی از تعریف اعضا با یک اصلاح کننده **داخلی محافظت شده** در زبان برنامه نویسی C# آورده شده است.

```
class User
{
    protected internal string Name;
    protected internal string Location;
    protected internal int Age;
    protected internal void GetUserDetails()
    {
        Console.WriteLine("Name: {0}", Name);
        Console.WriteLine("Location: {0}", Location);
        Console.WriteLine("Age: {0}", Age);
    }
}

class Program
{
    static void Main(string[] args)
    {
        User u = new User();
        u.Name = "Suresh Dasari";
        u.Name = "Suresh Dasari";
        u.Location = "Hyderabad";
        u.Age = 32;
        u.GetUserDetails();
    }
}
```



```

Console.WriteLine("\nPress Enter Key to Exit..");
Console.ReadLine();
}
}

```

اگر به مثال بالا توجه کنید، ما یک کلاس User را با متغیرها و متدهای مورد نیاز با استفاده از دسترسی داخلی محافظت شده تعریف کرده ایم و سعی می کنیم به آن متغیرها و متد در کلاس دیگری با مرجع شیء کلاس User دسترسی پیدا کنیم .

سطح دسترسی private protected :

در سی شارپ، سطح دسترسی محافظت شده خصوصی از نسخه ۷/۲ در دسترس است . برای تعیین اینکه دسترسی محدود به کلاس یا انوعی است که از کلاس در مجموعه جاری مشتق شده اند استفاده می شود . آن عضو را می توان با کد در همان کلاس یا یک کلاس مشتق شده در مجموعه کلاس پایه دسترسی داشت.

در زیر مثالی از تعریف اعضا با یک اصلاح کننده محافظت شده خصوصی در زبان برنامه نویسی C# آورده شده است.

```

class User
{
    private protected string Name;
    private protected string Location;
    private protected int Age;
    private protected void GetUserDetails()
    {
        Console.WriteLine("Name: {0}", Name);
        Console.WriteLine("Location: {0}", Location);
    }
}

```

```

        Console.WriteLine("Age: {0}", Age);
    }
}

class Program: User
{
    static void Main(string[] args)
    {
        User u = new User();
        Program p = new Program();
        // Compiler Error
        // protected members can only accessible with derived classes
        //u.Name = "Suresh Dasari";
        p.Name = "Suresh Dasari";
        p.Location = "Hyderabad";
        p.Age = 32;
        p.GetUserDetails();
        Console.WriteLine("\nPress Enter Key to Exit..");
        Console.ReadLine();
    }
}

```

اگر به مثال بالا توجه کنید، ما با استفاده از مرجع **کلاس** مشتق شده (**Program**) به اعضای **محافظت شده خصوصی کلاس** پایه **کاربر** دسترسی داریم . اگر ارث بری را حذف کنیم، یک خطای زمان کامپایل دریافت می کنیم زیرا سعی می کنیم به اعضای **محافظت شده خصوصی** با مرجع کلاس پایه **کاربر** به جای **کلاس** مشتق شده (**Program**) دسترسی پیدا کنیم.

OOO - C# چیست؟

OOO مخفف عبارت Object-Oriented Programming است.

برنامه نویسی رویه ای در مورد نوشتن رویه ها یا روش هایی است که عملیات روی داده ها را انجام می دهند، در حالی که برنامه نویسی شی گرا در مورد ایجاد اشیایی است که حاوی داده ها و روش ها هستند.

برنامه نویسی شی گرا چندین مزیت نسبت به برنامه نویسی رویه ای دارد:

- OOO سریعتر و آسانتر برای اجرا است
- OOO یک ساختار واضح برای برنامه ها فراهم می کند
- OOO به رعایت اصل DO NOT DRY کد سی شارپ یعنی "تکرار نکنید" کمک می کند و نگهداری، اصلاح و اشکال زدایی کد را آسان تر می کند.
- OOO امکان ایجاد برنامه های کاربردی قابل استفاده مجدد را با کد کمتر و زمان توسعه کوتاه تر ایجاد می کند

نکته :

اصل "تکرار نکنید (DRY)" در مورد کاهش تکرار کد است. شما باید کدهایی را که برای برنامه رایج هستند استخراج کنید و آنها را در یک مکان قرار دهید و به جای تکرار از آنها استفاده مجدد کنید.

۱. کپسوله سازی (Encapsulation)

Encapsulation فرآیندی است که اعضای یک شیء را به یک واحد متصل می کند. در سی شارپ، class مثالی برای کپسوله سازی است، زیرا انواع مختلفی از اعضای داده و توابع عضو را در یک واحد ترکیب می کند.

به طور کلی، در سی شارپ از کپسوله سازی برای جلوگیری از تغییر تصادفی کد (داده ها) از دنای بیرونی یک شیء استفاده می شود. در سی شارپ، با تعریف فیلدهای کلاس با property ها، می توانیم از داده ها در برابر خرابی تصادفی محافظت کنیم.

اگر فیلدهای کلاس را با خصوصیات تعریف کنیم ، کلاس کپسوله شده به ما اجازه دسترسی مستقیم به فیلدها را نخواهد داد. در عوض، ما باید از توابع گیرنده و تنظیم کننده برای خواندن یا نوشتن داده ها بر اساس نیازهای خود استفاده کنیم.

در زیر مثالی از تعریف کلاس **encapsulation** با استفاده از ویژگی ها با دسترسی های **get** و **set** آورده شده است :

```
class User
{
    private string location;
    private string name;
    public string Location
    {
        get
        {
            return location;
        }
        set
        {
            location = value;
        }
    }
    public string Name
    {
        get
        {
            return name;
        }
        set
        {
            name = value;
        }
    }
}
```

```

    }
}
}
class Program
{
    static void Main(string[] args)
    {
        User u = new User();
        // set accessor will invoke
        u.Name = "Suresh Dasari";
        // set accessor will invoke
        u.Location = "Hyderabad";
        // get accessor will invoke
        Console.WriteLine("Name: " + u.Name);
        // get accessor will invoke
        Console.WriteLine("Location: " + u.Location);
        Console.WriteLine("\nPress Enter Key to Exit..");
        Console.ReadLine();
    }
}

```

کد بالا را مشاهده کنید، متغیرها را با دسترسی خصوصی تعریف کرده ایم و آن متغیرها را به صورت عمومی با استفاده از ویژگی های **get** و **set** در معرض دید قرار داده ایم. اگر می خواهید تغییراتی در متغیرهای تعریف شده ایجاد کنید، می توانیم آن را با استفاده از ویژگی ها با دسترسی های **get** و **set** انجام دهیم.

۲. چند شکلی

Polymorphism به معنای "شکل های متعدد" است و زمانی اتفاق می افتد که ما کلاس های زیادی داشته باشیم که به واسطه ارث بری به یکدیگر مرتبط هستند.

همانطور که دیدید ؛ **وراثت به** ما این امکان را می دهد که فیلدها و متدها را از کلاس دیگری به ارث ببریم. **پلی مورفیزم** از آن متدها برای انجام وظایف مختلف استفاده می کند. این به ما امکان می دهد یک عمل واحد را به روش های مختلف انجام دهیم.

به عنوان مثال، یک کلاس پایه به نام **Animal** را در نظر بگیرید که متدی به نام **animalSound()** دارد. طبقات مشتق شده از حیوانات می تواند خوک ها، گربه ها، سگ ها، پرندگان باشد - و آنها همچنین صدای منحصر به فرد خود را دارند (گاو ماء می کند و گربه میو می کند و غیره):

```
class Animal // Base class (parent)
{
    public void animalSound()
    {
        Console.WriteLine("The animal makes a sound");
    }
}

class Pig : Animal // Derived class (child)
{
    public void animalSound()
    {
```

```
Console.WriteLine("The pig says: wee wee");
}
}

class Dog : Animal // Derived class (child)
{
    public void animalSound()
    {
        Console.WriteLine("The dog says: bow wow");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Animal myAnimal = new Animal(); // Create a Animal object

        Animal myPig = new Pig(); // Create a Pig object

        Animal myDog = new Dog(); // Create a Dog object
    }
}
```

```
myAnimal.animalSound();

myPig.animalSound();

myDog.animalSound();

}

}
```

خروجی به شکل زیر خواهد بود:

The animal makes a sound
The animal makes a sound
The animal makes a sound

نه خروجی ای که به دنبالش بودم این نیست!

خروجی مثال بالا احتمالاً آن چیزی نبود که انتظار داشتید. دلیل آن این است که متد کلاس پایه، زمانی که نام یکسانی دارند، متد کلاس مشتق شده را لغو می کند.

با این حال، سی شارپ با افزودن کلمه کلیدی **virtual** به متد داخل کلاس پایه و با استفاده از کلمه کلیدی **override** برای هر متد کلاس مشتق شده، گزینه ای را برای لغو متد کلاس پایه فراهم می کند:

```
class Animal // Base class (parent)
{
    public virtual void animalSound()
    {
        Console.WriteLine("The animal makes a sound");
    }
}
```



```
class Pig : Animal // Derived class (child)
{
    public override void animalSound()
    {
        Console.WriteLine("The pig says: wee wee");
    }
}
```

```
class Dog : Animal // Derived class (child)
{
    public override void animalSound()
    {
        Console.WriteLine("The dog says: bow wow");
    }
}
```

```
class Program
{
    static void Main(string[] args)
```

```
{  
  
    Animal myAnimal = new Animal(); // Create a Animal object  
  
    Animal myPig = new Pig(); // Create a Pig object  
  
    Animal myDog = new Dog(); // Create a Dog object  
  
  
    myAnimal.animalSound();  
  
    myPig.animalSound();  
  
    myDog.animalSound();  
  
}  
}
```

خروجی به این شکل خواهد بود:

The animal makes a sound

The pig says: wee wee

The dog says: bow wow

چرا و چه زمانی از "چند شکلی" استفاده کنیم؟

چند ریختی برای قابلیت استفاده مجدد کد مفید است یعنی می توانید از فیلدها و متدهای یک کلاس موجود در هنگام ایجاد یک کلاس جدید استفاده مجدد کنید.

۳. وراثت (Inheritance)

وراثت یکی از مفاهیم اولیه برنامه نویسی شی گرا (OOP) است و برای به ارث بردن خصوصیات از یک کلاس (پایه) به کلاس دیگر (فرزند) استفاده می شود.

وراثت ما را قادر می سازد تا با به ارث بردن خصوصیات از کلاس های دیگر برای استفاده مجدد، گسترش و اصلاح رفتار سایر اعضای کلاس بر اساس نیازهای خود، یک کلاس جدید ایجاد کنیم.

در وراثت به کلاس اصلی ، کلاس والد یا پدر گفته می شود و به کلاس هایی که از آن کلاس ارث بری می کنند کلاس فرزند یا کلاس مشتق شده می گویند.

ساختار ارث بری در زبان سی شارپ بصورت زیر است :

```
<access_modifier> class <base_class_name>
{
    // Base class Implementation
}
```

```
<access_modifier> class <derived_class_name> : <base_class_name>
{
    // اجرای کلاس مشتق شده
}
```

مثال زیر را در نظر بگیرید :

```
public class X
{
    public void GetDetails()
    {
        // Method implement
    }
}
```

```

public class Y: X
{
    // class
    your application }
class Program
{
    static void Main( string [] args)
    {
        Y y = new Y();
        y GetDetails ();
    }
}

```

اگر به مثال بالا توجه کنید، یک کلاس "X" را با متدی به نام "GetDetails" تعریف کردیم و کلاس "Y" از کلاس "X" به ارث بری می کند. پس از آن، با استفاده از نمونه ای از کلاس مشتق شده «Y»، متد «GetDetails» را فراخوانی می کنیم.

در سی شارپ، نمی توان [سازنده های](#) کلاس پایه را در کلاس مشتق شده به ارث برد. دسترسی سایر اعضای کلاس پایه نیز به [سطح دسترسی](#) بستگی دارد که برای تعریف آن اعضا در یک [کلاس](#) پایه استفاده می کنیم

وراثت چند سطحی سی شارپ

به طور کلی، سی شارپ تنها از **وراثت واحد** پشتیبانی می کند، به این معنی که یک [کلاس](#) فقط می تواند از یک [کلاس](#) پایه ارث بری کند. با این حال، در سی شارپ وراثت انتقالی است و به شما این امکان را می دهد که یک وراثت سلسله مراتبی را برای مجموعه ای از انواع تعریف کنید و به آن وراثت چند سطحی می گویند.

```

public class A
{
    // Implementation

```

```

}
public class B: A
{
// Implementation
}
public class C: B
{
// Implementation
}

```

اگر قطعه کد بالا را مشاهده کنید، کلاس **C** از کلاس **B**، و کلاس **B** از کلاس **A** مشتق شده است، سپس کلاس **C** اعضای اعلام شده در کلاس **B** و کلاس **A** را به ارث می برد. به این صورت است که می توانیم وراثت **چند سطحی** را در برنامه های خود پیاده سازی کنیم.

C# مثال وراثت چند سطحی

در زیر مثالی از پیاده سازی **وراثت چند سطحی** در زبان برنامه نویسی C# آورده شده است.

```

public class A
{
    public string Name;
    public void GetName()
    {
        Console.WriteLine("Name: {0}", Name);
    }
}
public class B: A
{
    public string Location;
    public void GetLocation()
    {
        Console.WriteLine("Location: {0}", Location);
    }
}

```

```

    }
}
public class C: B
{
    public int Age;
    public void GetAge()
    {
        Console.WriteLine("Age: {0}", Age);
    }
}
class Program
{
    static void Main(string[] args)
    {
        C c = new C();
        c.Name = "Suresh Dasari";
        c.Location = "Hyderabad";
        c.Age = 32;
        c.GetName();
        c.GetLocation();
        c.GetAge();
        Console.WriteLine("\nPress Any Key to Exit..");
        Console.ReadLine();
    }
}

```

اگر به مثال بالا توجه کنید، ما سه کلاس (A , B , C) را پیاده سازی کردیم و کلاس C از کلاس B و کلاس B از کلاس A مشتق شده است .

با اجرای یک وراثت چند سطحی، کلاس C می تواند اعضای اعلام شده در کلاس B و کلاس A را به ارث ببرد .

C# وراثت چندگانه

همانطور که گفته شد، سی شارپ فقط از **وراثت واحد** پشتیبانی می کند، به این معنی که یک [کلاس](#) فقط می تواند از یک [کلاس](#) پایه ارث بری کند . اگر بخواهیم یک کلاس را از چندین کلاس پایه به ارث ببریم، خطاهای زمان کامپایل را دریافت خواهیم کرد.

به عنوان مثال، اگر کلاس **C** بخواهد از کلاس **A** و **B** به طور همزمان ارث بری کند، یک خطای زمان کامپایل دریافت خواهیم کرد زیرا وراثت چندگانه در C# مجاز نیست.

```
public class A
{
    // Implementation
}
public class B
{
    // Implementation
}
public class C: A, B
{
    // Implementation
}
```

اگر قطعه کد بالا را مشاهده کنید، کلاس **C** در تلاش است تا [خصوصیات](#) را از هر دو کلاس **A** و **B** به طور همزمان به ارث ببرد ، که منجر به خطاهای زمان کامپایل مانند " کلاس **C** نمی تواند چندین کلاس داشته باشد **A** و " **B** می شود.

همانطور که بحث شد، وراثت **چند سطحی** در سی شارپ پشتیبانی می شود، اما وراثت **چندگانه** پشتیبانی نمی شود. اگر می خواهید وراثت **چندگانه** را در سی شارپ پیاده سازی کنید ، می توانیم با استفاده از [اینترفیس ها](#) به این هدف برسیم.

چرا و چه زمانی از "ارث بری" استفاده کنیم؟

-برای قابلیت استفاده مجدد کد مفید است: از فیلدها و متدهای یک کلاس موجود در هنگام ایجاد یک کلاس جدید استفاده مجدد کنید.

۴. انتزاع (Abstraction)

انتزاع داده فرآیند پنهان کردن جزئیات خاص و نشان دادن تنها اطلاعات ضروری به کاربر است. **abstract** کلمه کلیدی برای کلاس ها و متدها مورد استفاده قرار می گیرد.

- **کلاس Abstract** : یک کلاس محدود است که نمی توان از آن برای ایجاد اشیا استفاده کرد (برای دسترسی به آن باید از کلاس دیگری به ارث برده شود).

- **متد انتزاعی** : فقط در کلاس انتزاعی قابل استفاده است و بدنه ندارد. بدنه توسط کلاس مشتق شده (به ارث رسیده از) ارائه می شود.

یک کلاس انتزاعی می تواند هم متدهای انتزاعی و هم متدهای معمولی داشته باشد:

```
abstract class Animal
{
    public abstract void animalSound();

    public void sleep()
    {
        Console.WriteLine("Zzz");
    }
}
```



```
}
```

در مثال بالا، امکان ایجاد یک شی از کلاس Animal وجود ندارد:

```
Animal myObj = new Animal(); // Will generate an error (Cannot create an instance of the abstract class or interface 'Animal')
```

برای دسترسی به کلاس انتزاعی باید از کلاس دیگری به ارث برده شود. بیا یک کلاس Animal را که در [Polymorphism](#) استفاده کردیم به یک کلاس انتزاعی تبدیل کنیم.

نکته : از بخش [Inheritance](#) به یاد داشته باشید که ما از نماد **:** برای ارث بردن از یک کلاس استفاده می کنیم و از کلمه کلیدی **override** برای لغو متد کلاس پایه استفاده می کنیم.

```
// Abstract class

abstract class Animal
{
    // Abstract method (does not have a body)
    public abstract void animalSound();

    // Regular method
    public void sleep()
    {
        Console.WriteLine("Zzz");
    }
}
```

```
// Derived class (inherit from Animal)
```

```
class Pig : Animal
{
    public override void animalSound()
    {
        // The body of animalSound() is provided here
        Console.WriteLine("The pig says: wee wee");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Pig myPig = new Pig(); // Create a Pig object
        myPig.animalSound(); // Call the abstract method
        myPig.sleep(); // Call the regular method
    }
}
```

چرا و چه زمانی از کلاس ها و متد های انتزاعی استفاده کنیم؟

زمانی که نیاز به دستیابی به امنیت و پنهان کردن جزئیات خاصی داشته باشیم از abstraction استفاده می کنیم و فقط جزئیات مهم یک شی را نشان می دهیم.

اصول طراحی شی گرا SOLID چیست؟

SOLID دربرگیرنده اصولی در برنامه نویسی شیء گرایی است که در اوایل سال ۲۰۰۰ توسط مهندسی به نام Robert Martin ابداع شد که تحت عنوان Uncle Bob یا «عمو باب» نیز شناخته می شود. وقتی این اصول به درستی در کنار یکدیگر به کار گرفته شوند، این امکان را به برنامه نویس یا توسعه دهنده می دهند تا با سهولت بیشتری دست به توسعه نرم افزار بزند مضاف بر اینکه به کارگیری این اصول امکانی را به برنامه نویسان خواهد داد تا با رویکردی چابک به توسعه نرم افزارهای خود پرداخته، از مرتکب شدن اشتباهات کوچک جلوگیری کنند و در صورت نیاز هم به سادگی اقدام به بازنویسی کدهای خود کنند (در فصل آینده، با مفهوم برنامه نویسی چابک بیشتر آشنا خواهیم شد). به طور کلی، **SOLID** مخفف اصطلاحات زیر است:

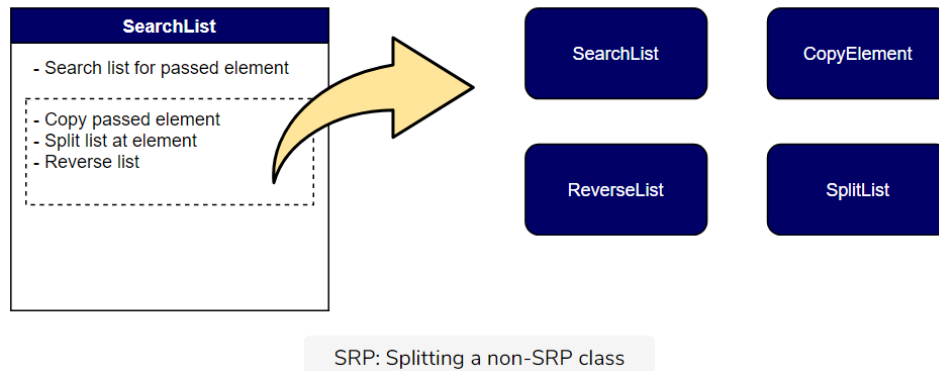
- **S - Single-responsibility principle** اصل مسئولیت واحد یا اصل تک مسئولیتی بودن کلاس ها
- **O - Open-closed principle** اصل باز/بسته بودن کلاس ها
- **L - Liskov substitution principle** اصل جانشینی لیسکوف
- **I - Interface segregation principle** اصل تفکیک اینترفیس
- **D - Dependency Inversion Principle** اصل انطباق پذیری

۱. اصل مسئولیت واحد یا اصل تک مسئولیتی بودن کلاس ها S - Single-responsibility principle

این قانون که به طور خلاصه SRP نیز نامیده می شود، حاکی از آن است که یک کلاس باید صرفاً یک وظیفه بیشتر نداشته باشد که در این صورت، کلاس ها فقط و فقط به خاطر ایجاد تغییر در وظیفه ای که انجام می دهند دستخوش تغییر خواهند شد نه چیز دیگر! کلاس ها می توانند فیچرهای مختلفی داشته

باشند اما تمامی آن‌ها باید مربوط به یک حوزه بوده و مرتبط به هم باشند که در نهایت با محترم شمردن چنین قانونی، برنامه‌نویسان دیگر قادر نخواهند بود تا کلاس‌های اصطلاحاً همه‌فن‌حریف بنویسند.

S: Single-responsibility principle



به مثال زیر دقت کنید :

```
public class Employee
{
    public int Employee_Id { get; set; }
    public string Employee_Name { get; set; }

    /// <summary>
    /// This method used to insert into employee table
    /// </summary>
    /// <param name="em">Employee object</param>
    /// <returns>Successfully inserted or not</returns>
    public bool InsertIntoEmployeeTable(Employee em)
    {
        // Insert into employee table.
    }
}
```

```

        return true;
    }
    /// <summary>
    /// Method to generate report
    /// </summary>
    /// <param name="em"></param>
    public void GenerateReport(Employee em)
    {
        // Report generation with employee data using crystal report.
    }
}

```

کلاس Employee در واقع دو کار انجام می دهد یعنی هم وظیفه ی ثبت اطلاعات کارمندان را در بانک اطلاعاتی برعهده دارد و هم متدی برای تهیه ی گزارش از کارمندان دارد. در واقع این کلاس نباید عملیات تولید گزارش را هم انجام دهد ، فرض کنید پس از تحویل پروژه به کارفرما پس از مدتی ، کارفرما از شما می خواهد که گزارشی از کارمندان را در قالب excel ارائه دهید ، آنگاه شما چکار می کنید؟ خب طبیعی است که باید به این کلاس مراجعه کرده و یک متدی دیگر را برای تهیه گزارش با فرمت جدید به آن اضافه نمائید، پس باید کدهای این کلاس را تغییر دهید و این اصلا کار خوبی نیست! بنابراین با توجه به اصل اول SOLID که باید هر کلاس یک وظیفه داشته باشد، کد فوق را اینگونه تغییر می دهیم :

```

public class ReportGeneration
{
    /// <summary>
    /// Method to generate report
    /// </summary>
    /// <param name="em"></param>
    public void GenerateReport(Employee em)
    {
        // Report reneration with employee data.
    }
}

```

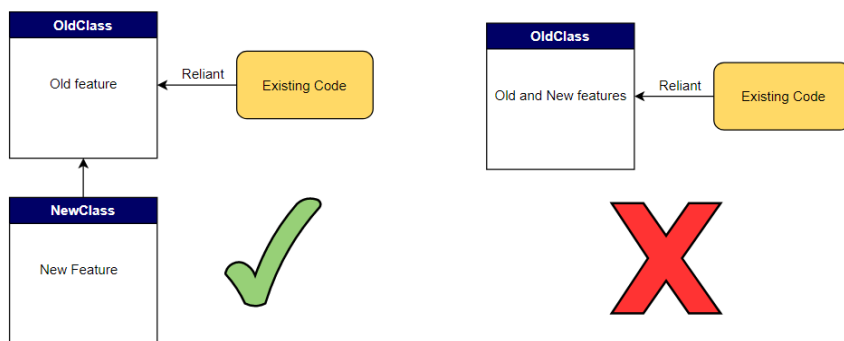
```
}  
}
```

این کلاس فقط وظیفه ی تولید گزارش را برعهده دارد و به صورت مستقل از کلاس Employee عمل می کند.

۲. اصل باز/بسته Open-closed Principle

هر کلاسی باید برای توسعه یافتن قابلیت هایش اصطلاحاً Open بوده و دست برنامه نویس برای افزودن فیچرهای جدید به آن باز باشد اما اگر وی خواست تا تغییری در کلاس ایجاد کند، چنین امکان باید Closed بوده و او اجازه چنین کاری را نداشته باشد. فرض کنیم نرم افزاری نوشته ایم که دارای چندین کلاس مختلف است و نیازهای اپلیکیشن مان را مرتفع می سازند اما به جایی رسیده ایم که نیاز داریم قابلیت های جدید به برنامه خود بیفزاییم. بر اساس این قانون، دست مان برای تغییر یا بهتر بگوییم افزودن فیچرهای جدید به کلاس مد نظر باز است در حالی که این قابلیت های جدید باید در قالب افزودن کدهای جدید صورت پذیرد نه ریفکتور کردن و تغییر کدهای قبلی!

O: Open-closed principle



OCP: Modifying OldClass instead of extending

برای روشن تر شدن این مسأله مثالی می زنیم. کلاس زیر را در نظر بگیرید:

```

public class ReportGeneration
{
    /// <summary>
    /// Report type
    /// </summary>
    public string ReportType { get; set; }

    /// <summary>
    /// Method to generate report
    /// </summary>
    /// <param name="em"></param>
    public void GenerateReport(Employee em)
    {
        if (ReportType == "CRS")
        {
            // Report generation with employee data in Crystal Report.
        }
        if (ReportType == "PDF")
        {
            // Report generation with employee data in PDF.
        }
    }
}

```

به نظر شما چه اشکالی در طراحی این کلاس وجود دارد؟! بله درست حدس زدید! وجود if های زیاد اشکال طراحی است ، اما چرا؟ به دلیل اینکه مثلاً اگر شما بعدها بخواهید انواع دیگر گزارش را ایجاد کنید ، باید کدهای این کلاس را مرتباً تغییر دهید و if های بیشتری بنویسید که این خلاف اصل دوم می باشد. پس باید چکار کنیم ؟ به کدهای زیر دقت کنید :

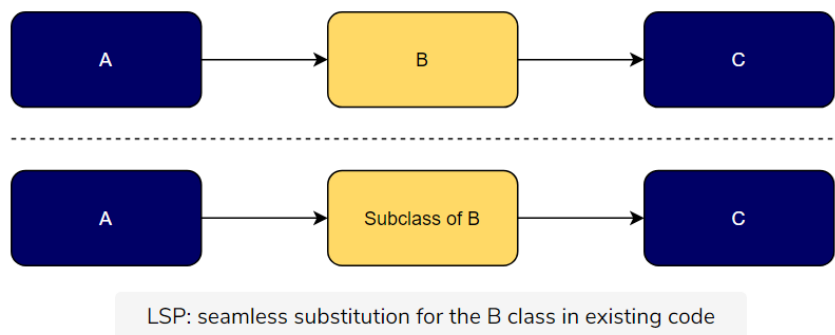
```
public class IReportGeneration
{
    /// <summary>
    /// Method to generate report
    /// </summary>
    /// <param name="em"></param>
    public virtual void GenerateReport(Employee em)
    {
        /// From base
    }
}
/// <summary>
/// Class to generate Crystal report
/// </summary>
public class CrystalReportGeneraion : IReportGeneration
{
    public override void GenerateReport(Employee em)
    {
        /// Generate crystal report.
    }
}
/// <summary>
/// Class to generate PDF report
/// </summary>
public class PDFReportGeneraion : IReportGeneration
{
    public override void GenerateReport(Employee em)
    {
        /// Generate PDF report.
    }
}
```


در این حالت از یک interface استفاده کردیم که متد ایجاد گزارش را برای آن تعریف کردیم حال به ازای هر نوع گزارشی که نیاز داشته باشیم یک کلاس جداگانه ای می نویسیم و از آن interface ارث بری می کنیم و کدهای مربوطه را داخل آن می نویسیم که نیاز به تغییر کد کلاس والد نباشد پس به جای آنکه قابلیت های مد نظر جدید را به کلاس والد بیفزاییم، نیاز خود را از طریق افزودن قابلیت های جدید در همان کلاس فرزند عملی می کنیم. به عبارتی، هرگز دست به تغییر کدهای موجود نزده و قانون Open/Closed را هم به رسمیت شناخته ایم به طوری که کلاس مد نظرمون برای توسعه باز است اما برای اعمال هر گونه تغییری بسته است.

۳. اصل جانشینی لیسکوف Liskov substitution principle

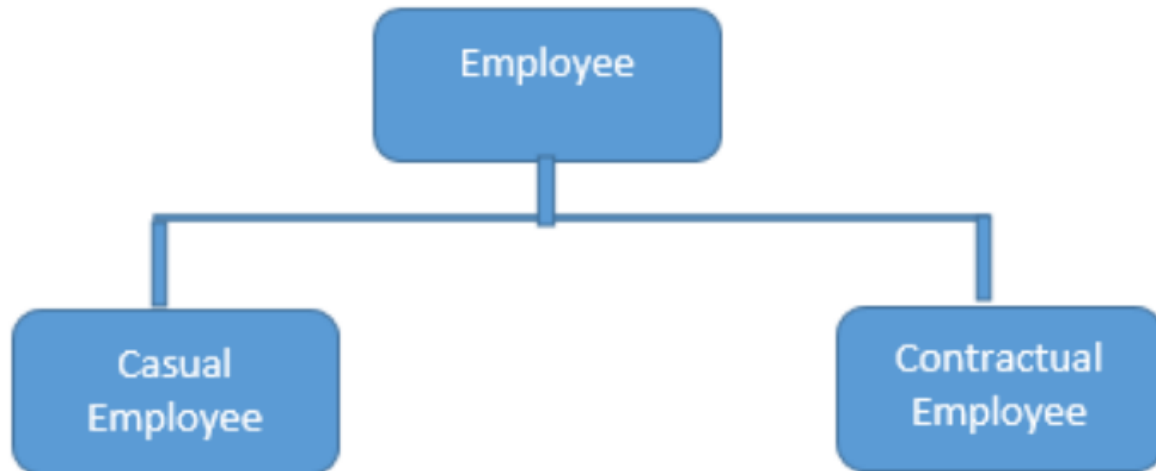
این اصل حاکی از آن است که کلاس های فرزند باید آن قدر کامل و جامع از کلاس والد خود ارث بری کرده باشند که به سادگی بتوان همان رفتاری که با کلاس والد می کنیم را با کلاس های فرزند نیز داشته باشیم حال که اگر در شرایطی قرار گرفتید که با خود گفتید کلاس فرزند می تواند تمامی کارهای کلاس والدش را انجام دهد به جزء برخی موارد خاص، اینجاست که این اصل از SOLID را نقض کرده اید.

L: Liskov substitution principle



این اصل ساده است اما درک آن بسیار مهم است. کلاس فرزند نباید تعریف و رفتار کلاس والد را از بین ببرد. حالا معنی این چیست!! خوب، اجازه دهید همان کارمند را مثال بزنم تا شما این اصل را بهتر درک

کنید، تصویر و کدهای زیر را بررسی کنید Employee یک کلاس والد است و کارمند عادی و قراردادی کلاسهای فرزند هستند که از کلاس کارمند ارث بری می کنند.



```
public abstract class Employee
{
    public virtual string GetProjectDetails(int employeeId)
    {
        return "Base Project";
    }
    public virtual string GetEmployeeDetails(int employeeId)
    {
        return "Base Employee";
    }
}
public class CasualEmployee : Employee
{
    public override string GetProjectDetails(int employeeId)
    {
        return "Child Project";
    }
}
```

```
}  
// May be for contractual employee we do not need to store the details into  
database.  
public override string GetEmployeeDetails(int employeeId)  
{  
    return "Child Employee";  
}  
}  
public class ContractualEmployee : Employee  
{  
    public override string GetProjectDetails(int employeeId)  
    {  
        return "Child Project";  
    }  
  
    // May be for contractual employee we do not need to store the details into  
database.  
    public override string GetEmployeeDetails(int employeeId)  
    {  
        throw new NotImplementedException();  
    }  
}
```

ظاهرا همه چیز درست است ، اما به کد زیر دقت کنید که اصل LSP را نقض می کند

```
List<Employee> employeeList = new List<Employee>();
employeeList.Add(new ContractualEmployee());
employeeList.Add(new CasualEmployee());
foreach (Employee e in employeeList)
{
    e.GetEmployeeDetails(1245);
}
```

خب فکر کنم که مشکل را حدس زده باشید! بله درست است چون در کلاس **contractual employee** برای متد **GetEmployeeDetails** کدی نوشته نشده است پس سیستم یک خطا می دهد. حال ببینیم راه حل این مشکل چیست؟

باید همه چیز را در ۲ interface مختلف، **IPProject** و **IEmployee** و بر اساس نوع کارمند پیاده سازی می کنیم :

```
public interface IEmployee
{
    string GetEmployeeDetails(int employeeId);
}

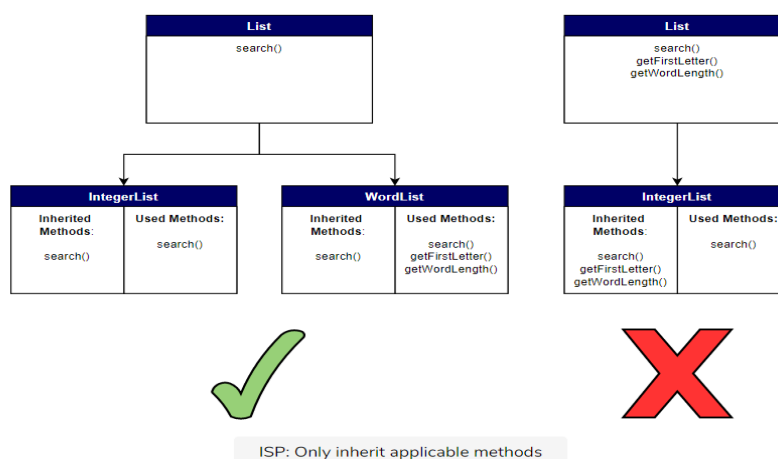
public interface IPProject
{
    string GetProjectDetails(int employeeId);
}
```

حال کلاس **contractual employee** را فقط از **IEmployee** ارث بری می کنیم و کلاس **CasualEmployee** را از **IEmployee** و **IPProject** ارث بری و متدهای مورد نظر را پیاده سازی می کنیم.

۴. اصل تفکیک اینترفیس Interface segregation principle

پیش از این هم گفتیم که اینترفیس‌ها فقط مشخص می‌کنند که یک کلاس از چه متدهایی حتماً باید برخوردار باشد. در همین راستا و بر اساس این قانون، چندین اینترفیس تک‌منظوره به مراتب بهتر است از یک اینترفیس چندمنظوره است به طوری که اگر یک اینترفیس چندمنظوره کامل و جامع داشته باشیم و سایر کلاس‌های ما از آن اصطلاحاً **implements** شوند، در چنین صورتی ممکن است برخی خصوصیات، متدها و رفتارها را به برخی کلاس‌هایی که اصلاً نیازی به آن‌ها ندارند تحمیل کنیم اما اگر از چندین اینترفیس تخصصی استفاده کنیم، به سادگی می‌توانیم از هر اینترفیسی که نیاز داشته باشیم در کلاس‌های مد نظر خود استفاده نماییم و در صورتی هم کلاسی وجود داشت که نیاز به استفاده از چندین اینترفیس مختلف داشت، دست ما باز خواهد بود تا آن کلاس را از چندین اینترفیس **implements** کنیم.

I: Interface segregation principle



Interface زیر را در نظر بگیرید :

```
public interface IEmployee
{
    bool AddEmployeeDetails();
}
```

حال تمام انواع کلاس کارمند این رابط را برای ذخیره داده ها به ارث خواهند برد. این خوبه درسته؟ حال فرض کنید کارفرما روزی به شما بگوید که می خواهد فقط داده های کارمندان دائمی را بخواند و نه همه ی کارمندان را ، آن وقت چه کاری انجام خواهید داد ؟، فقط یک متد را به این رابط اضافه می کنید؟

```
public interface IEmployeeDatabase
{
    bool AddEmployeeDetails();
    bool ShowEmployeeDetails(int employeeId);
}
```

اما الان داریم چیزی را تحمیل می کنیم ، ما کلاس کارمند غیردائمی را مجبور می کنیم که جزئیات خود را از پایگاه داده خوانده و نشان دهد. بنابراین، راه حل این است که این مسئولیت را به یک interface دیگر بسپارید.

```
public interface IAddOperation
{
    bool AddEmployeeDetails();
}

public interface IGetOperation
{
    bool ShowEmployeeDetails(int employeeId);
}
```

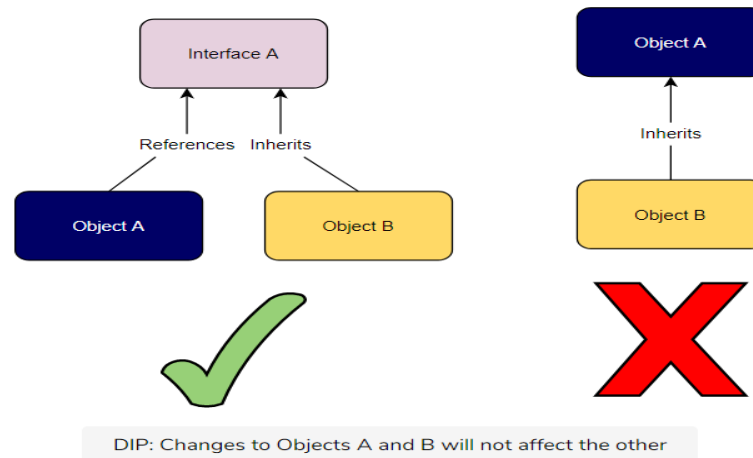
با این کار کلاس کارمند غیردائمی فقط IAddOperation را پیاده سازی می کند و کلاس کارمند دائمی هر دو رابط را پیاده سازی می کند که مطابق با اصل ISP می باشد.

۵. اصل انطباق پذیری Dependency Inversion Principle

درک این اصل از اصول پنج گانه SOLID کمی دشوارتر به نظر می رسد و برخی تازه کارها آن را با Dependency Injection اشتباه می گیرند. به طور خلاصه، در OOP باید تمام تلاش خود را به کار بندیم تا Dependency (وابستگی) را مابین کلاس ها، ماژول ها و آبجکت های سطح بالا با ماژول های

سطح پایین به حداقل برسانیم که با این کار، اعمال تغییرات در آینده به مراتب راحت تر صورت خواهد پذیرفت.

D: Dependency inversion principle



فرض کنید پس از ذخیره برخی جزئیات در پایگاه داده، یک سیستم اعلان داشته باشیم، کد زیر را در نظر بگیرید:

```
public class Email
{
    public void SendEmail()
    {
        // code to send mail
    }
}

public class Notification
{
    private Email _email;
    public Notification()
    {
```

```

        _email = new Email();
    }

    public void PromotionalNotification()
    {
        _email.SendEmail();
    }
}

```

اکنون کلاس Notification کاملاً به کلاس Email بستگی دارد، زیرا فقط یک نوع اعلان را ارسال می کند. اگه بخواهیم یک نوع دیگر از اعلان را مثل اس ام اس معرفی کنیم ، آنگاه ما باید سیستم اطلاع رسانی را نیز تغییر دهیم. و این را وابستگی می گویند حال چه کار کنیم که آن را برطرف نماییم؟ خوب، کد زیر را بررسی کنید:

```

public interface IMessenger
{
    void SendMessage();
}

public class Email : IMessenger
{
    public void SendMessage()
    {
        // code to send email
    }
}

public class SMS : IMessenger
{
    public void SendMessage()

```



```

    {
        // code to send SMS
    }
}
public class Notification
{
    private IMessenger _iMessenger;
    public Notification()
    {
        _ iMessenger = new Email();
    }
    public void DoNotify()
    {
        _ iMessenger.SendMessage();
    }
}

```

هنوز کلاس اعلان به کلاس ایمیل بستگی دارد. اکنون می‌توانیم از تزریق وابستگی استفاده کنیم تا بتوانیم آن را آزادانه به هم مرتبط نمائیم .

۳ نوع روش برای DI، تزریق سازنده (Constructor injection)، تزریق خواص (Property injection) و تزریق متد (method injection) وجود دارد.

روش اول :

Constructor Injection

```

public class Notification
{
    private IMessenger _iMessenger;
    public Notification(IMessenger pMessenger)
    {

```

```
        _ iMessenger = pMessenger;
    }
    public void DoNotify()
    {
        _ iMessenger.SendMessage();
    }
}
```

روش دوم :

Property Injection

```
public class Notification
{
    private IMessenger _iMessenger;

    public Notification()
    {
    }

    public IMessenger MessageService
    {
        private get;
        set
        {
            _ iMessenger = value;
        }
    }

    public void DoNotify()
    {
        _ iMessenger.SendMessage();
    }
}
```

```
}  
}
```

روش سوم :

Method Injection

```
public class Notification  
{  
    public void DoNotify(IMessenger pMessenger)  
    {  
        pMessenger.SendMessage();  
    }  
}
```

بنابراین، اصول SOLID به ما کمک می‌کند تا کدهایی بنویسیم که بسیار قابل نگهداری و کمتر مستعد خطا باشد.